COMS4995W30
Applied Machine Learning

Dr. Spencer W. Luo

Columbia University | Fall 2026

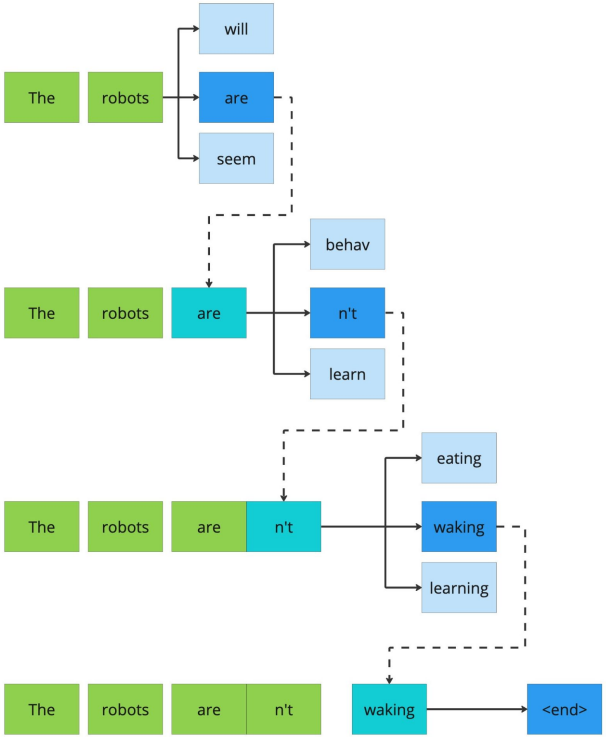


Transformers in Practice



Motivation

How does ChatGPT work in a nutshell? 🤔



Task: Next-Token Prediction 🎯



Given previous tokens, predict the next token.

The output is a probability distribution over the vocabulary:

p(next token | previous tokens)

There is one probability for each of the |V| vocabulary entries.

Each position is a multi-class classification problem.

Tokens 



Text is split into tokens

- Often subwords, not full words
- Better discrete units that capture reusable language patterns
- WordPiece example: “playing” splits into “play” and “##ing”

Each token is mapped to an integer ID via the tokenizer’s vocabulary

- Vocabulary size: tens to hundreds of thousands of tokens

Training Objective - Cross-Entropy Loss 📉



Since this is a multi-class problem, we use cross-entropy loss:

$$L = - \Sigma_t \log p(x_t \mid x_1, ..., x_{t-1})$$

It penalizes the model when it assigns low probability to the correct next token

Same loss as we used in earlier MLP sessions

- Only difference: inputs are sequential texts

Predict the next token

- This simple rule is the core principle of Language Modeling

First Attempt: Use an MLP 🧩



Feed the entire context representation into an MLP

MLP learns non-linear mappings, and predicts next tokens

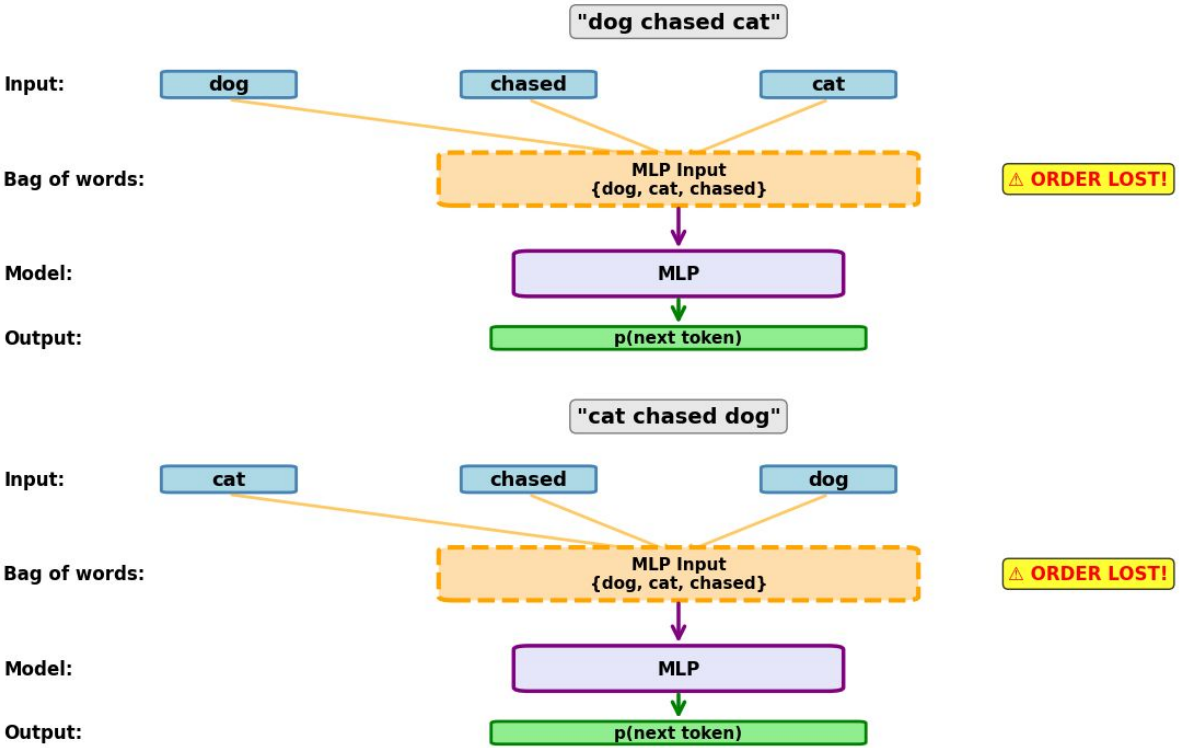
Pros ✅

- Learns flexible patterns and interactions between features

Cons ❌

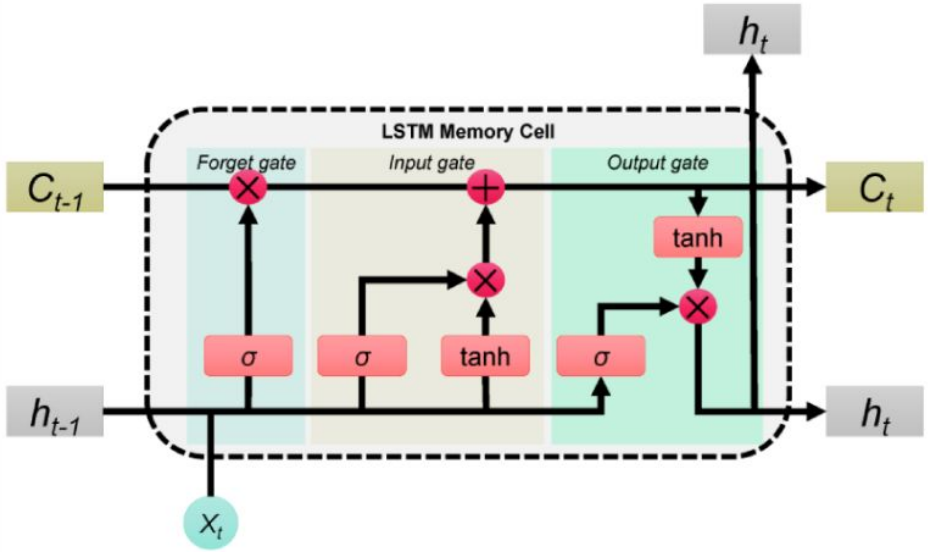
- *Bag-of-words loses order; concatenation has a fixed size*
- No built-in mechanism for selecting relevant context tokens

Order Matters



Add Order: Try RNN / LSTM 🚀

Recurrent Neural Networks / Long Short-Term Memory



Add Order: Try RNN / LSTM 🚀



Pros ✅

- Capture short-term context
- Great for small sequences

Cons ❌

- Training is slow and non-parallelizable
- Long-range dependencies fade (vanishing gradients)
- Inefficient for large-scale texts

What We Really Need ⚡



A neural network model that:

- Understands relationships between all tokens in a sequence
- Computes these relationships in parallel
- Learns which tokens matter most for the prediction

To Build Transformer 🤖



1️⃣ Self-Attention Mechanism

2️⃣ Residual Connection

3️⃣ Transformer Block



Self-Attention Mechanism

Context Changes a Token's Meaning



RIVER CONTEXT

"We rested on the bank of the river." — land beside water

FINANCIAL CONTEXT

"I deposited my savings at the bank." — financial institution

A token starts with an embedding; the surrounding sequence changes its contextual state.

Self-Attention ➡️



In next-token prediction, all the words come from the same sequence

Each word simply looks at the others in the same sentence to understand their contextual relationships

- This is called Self-Attention

It helps information flow among all words, allowing the model to understand context before predicting the next token

Core Flow 📄



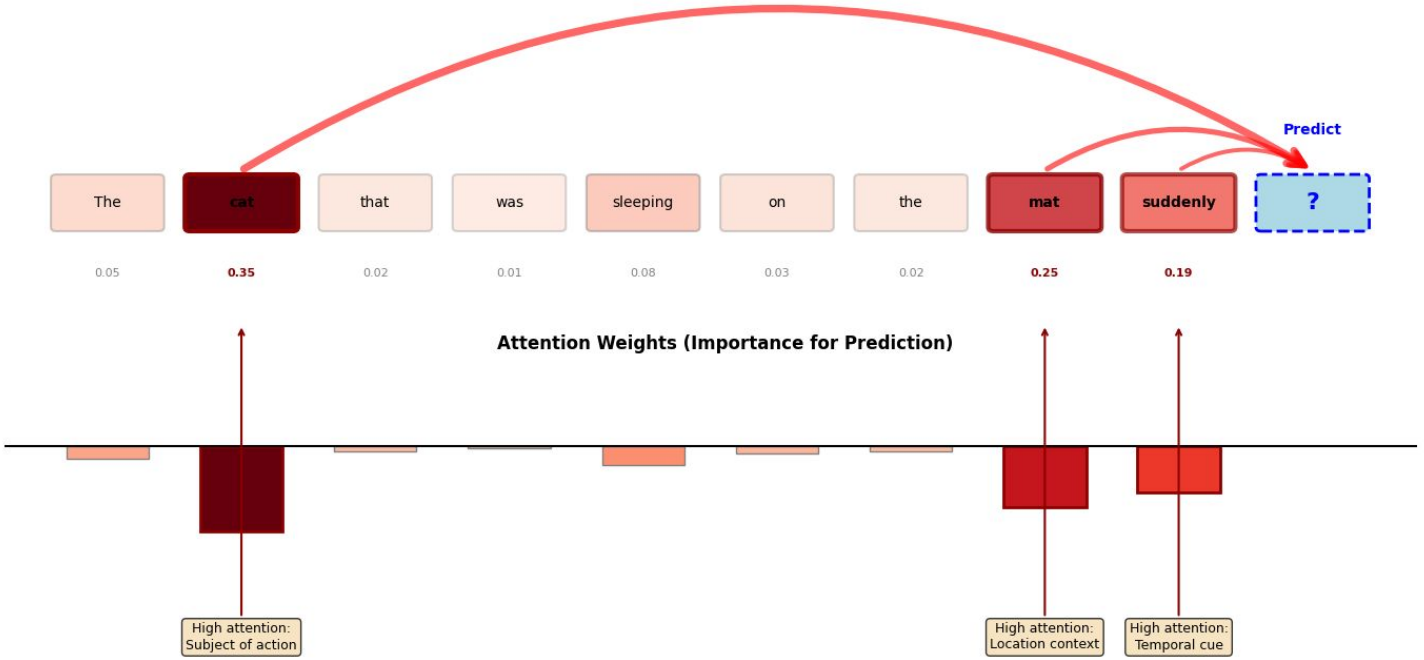
Each token asks: “Which other tokens should I pay attention to?”

The model computes similarity scores between tokens

These scores become weights that determine how much information to take from each token

Attention returns a weighted average of value vectors

Visualization 👀



Why Self-Attention Matters 🤔



Self-Attention allows information to flow among all words in a sentence

Each word updates its understanding based on what others mean

This creates rich, context-aware representations for every position

In next-token prediction, this helps the model understand the full context before guessing the next word

How does this actually happen inside the model?

Step 1 – Token Embeddings 🧱



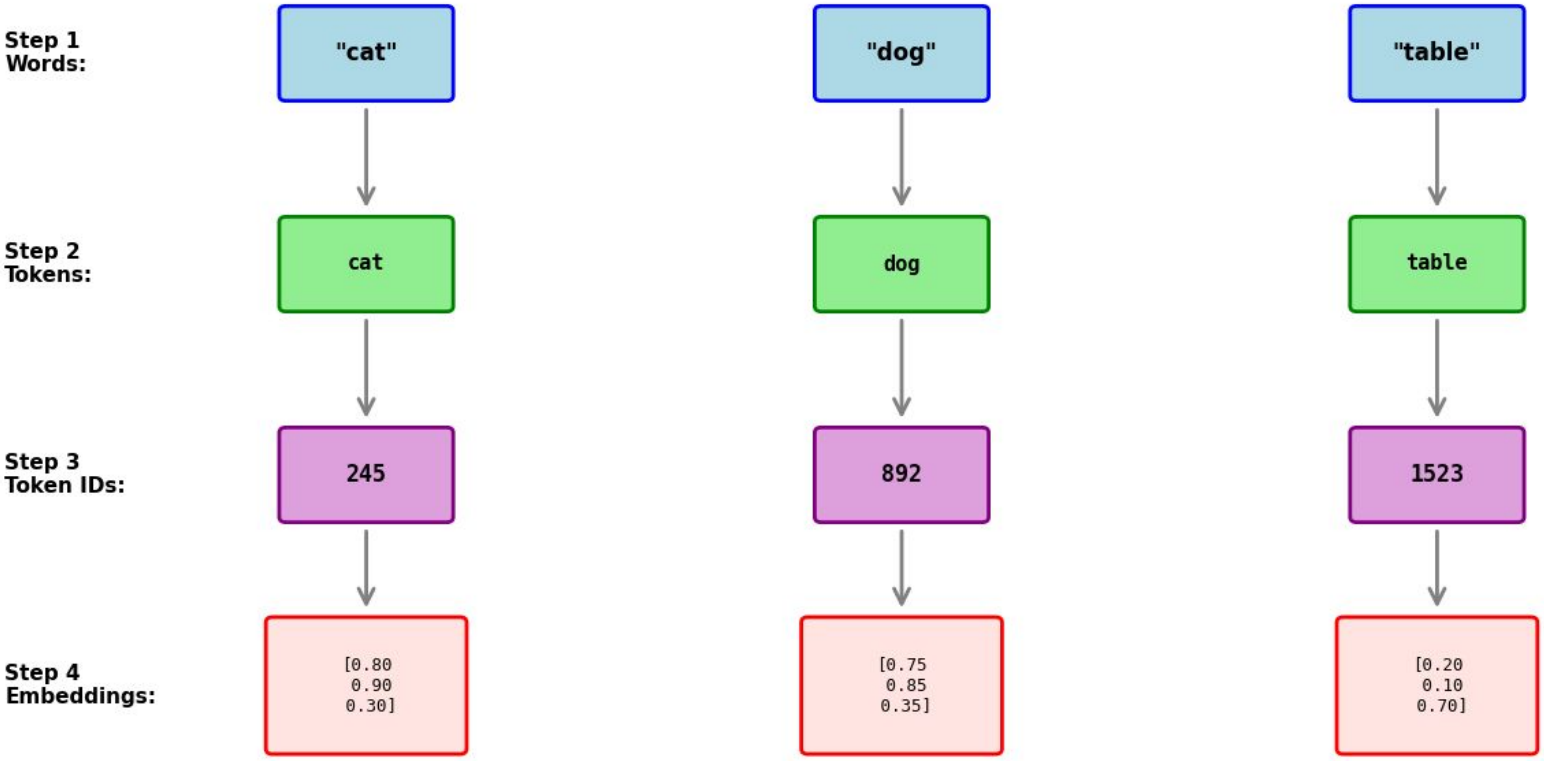
Computers cannot understand text directly

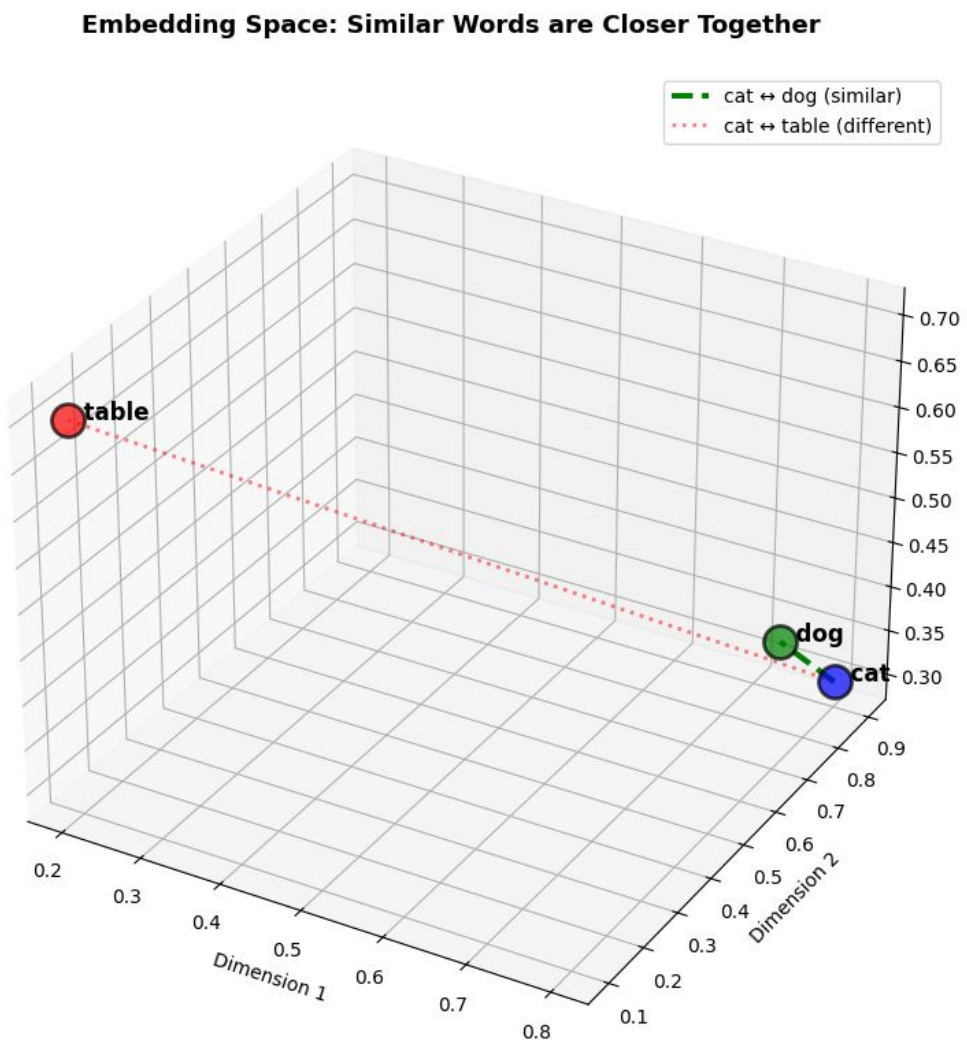
So each token (like “dog”, “cat”, “run”) is converted into a vector of numeric values

Ideally, these vectors shall capture semantic meaning

- Vectors of “cat” and “dog” are closer than the ones of “cat” and “table”

This vector form is called an embedding


Step 1
Words:
"cat"
"dog"
"table"
Step 2
Tokens:
cat
dog
table
Step 3
Token IDs:
245
892
1523
Step 4
Embeddings:
[0.80
0.90
0.30]
[0.75
0.85
0.35]
[0.20
0.10
0.70]
Computers need numbers, not text!
Word → Token → Token ID → Embedding (dense vector)


Embedding Space: Similar Words are Closer Together
cat ↔ dog (similar)
cat ↔ table (different)
table
dog
cat
Dimension 1
Dimension 2
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.1
0.9
0.70
0.65
0.60
0.55
0.50
0.45
0.40
0.35
0.30

Step 2 – Linear Projections

$$Q = XW_Q$$

3 token rows; 4 embedding columns.

Input states X W_Q **Q**

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} & x_{14} \\ x_{21} & x_{22} & x_{23} & x_{24} \\ x_{31} & x_{32} & x_{33} & x_{34} \end{bmatrix} \times \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix} = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

3×4 $\quad$ 4×1 $\quad$ 3×1

Toy example: map each token's 4 features to 1 query number.

Query, Key, and Value



Each token gets one query, one key and one value number.

TOKEN	Q · Query	K · Key	V · Value
Returns	q_1	k_1	v_1
require	q_2	k_2	v_2
a	q_3	k_3	v_3
	match request	key features	content to mix

Worked Example: Q, K and V

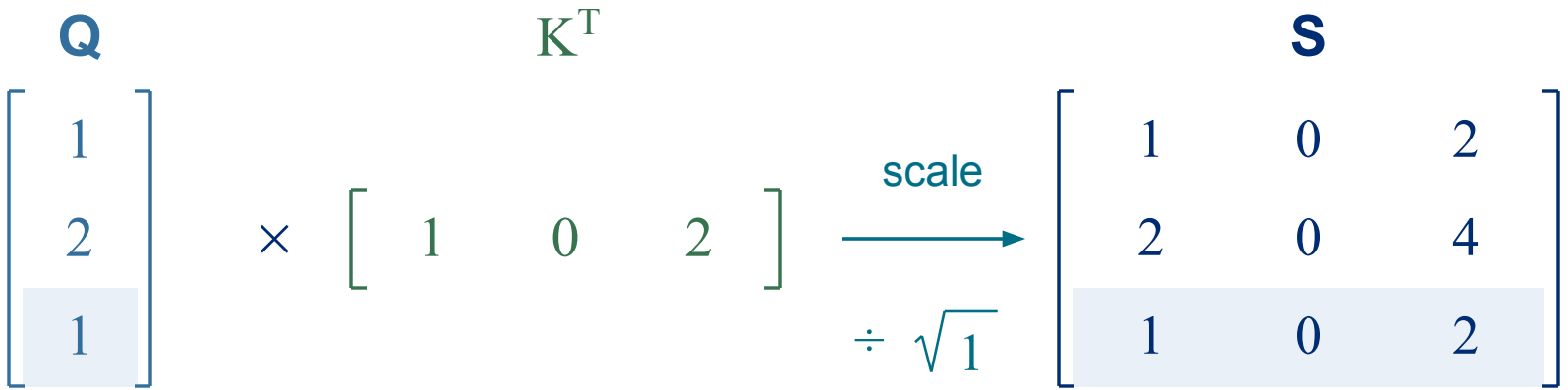
3 tokens; one number per token in each of Q, K and V.

position	Q	K	V
1	1	1	2
2	2	0	0
3	1	2	4
	3×1	3×1	3×1

Track row 3: $q_3 = 1$.

Step 3 – Compute Similarity Scores

Rows = query tokens; columns = key tokens.



$d_k = 1$, so the scale is 1.

3 × 3 token scores,
not probabilities.

Step 4 – Apply Causal Mask ⛓

In next-token prediction, the model must not peek at future words - it can only use what it has seen so far.

To enforce this rule, we use a causal mask:

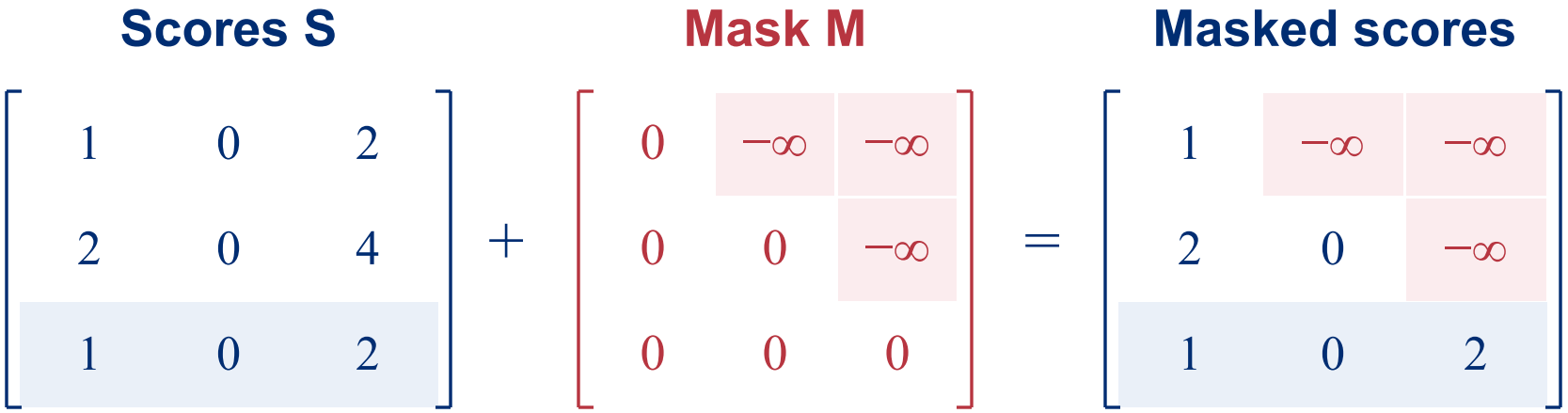
- Apply upper-triangular matrix → set to –∞ before softmax

This guarantees autoregressive generation - each token learns only from tokens up to that point, never ahead

At step t, the model can listen only to tokens 1 … t - but not beyond!

Step 4 – Apply the Causal Mask

Quick check: can the output at position 2 use value 3?



Mask: add 0 at positions 1…i; add −∞ at future positions.

Causal Masking – Which Triangle 🤔



Keep Mask (Allowed)

- Lower Triangle - tokens of positions ≤ i (past + current)

Blocked Mask (Forbidden)

- Upper Triangle -tokens of positions > i (future)

👀 PyTorch Implementation:

```
tgt_mask = torch.triu(torch.ones(T, T) * float('-inf'), diagonal=1)
```

🎤 Mock Interview

A decoder predicts the next word from a prompt. During training, which positions may each token attend to?

A) Every token in the training example
B) Only earlier positions and its own position
C) Only the immediately previous token
D) Only future positions

2 minutes with a partner. Choose one answer and explain why.

Step 5 – Normalize with Softmax 🌡️



Normalize the masked scores into attention weights:

Weights = softmax(masked scaled scores)

Each row becomes a probability distribution over the visible tokens

Scale the dot products by sqrt(d_k) before masking.

Each row sums to 1; these are weights over token positions.

Step 5 – Normalize with Softmax

Softmax turns each score row into weights that sum to 1.

Masked scores

1	$-\infty$	$-\infty$
2	0	$-\infty$
1	0	2

row-wise softmax →

Attention weights A

			row sum
1	0	0	= 1
0.881	0.119	0	= 1
0.245	0.090	0.665	= 1

Normalize over the 3 key tokens in each query row.

$A_{23} = 0$: position 2 cannot use v_3.

Step 6 – Weighted Sum of Values 📦

$$\text{Output} = \text{Weights} \cdot V$$

Each token representation = contextual blend of other tokens

Captures both short- and long-range dependencies

Fully parallelized across the sequence

Training processes positions in parallel; generation remains token by token.

Step 6 – Mix the Value Vectors

A (3 × 3) mixes V (3 × 1), producing H (3 × 1).

Weights A **Values V** **Output H**

$$\begin{bmatrix} 1 & 0 & 0 \\ 0.881 & 0.119 & 0 \\ 0.245 & 0.090 & 0.665 \end{bmatrix} \times \begin{bmatrix} 2 \\ 0 \\ 4 \end{bmatrix} = \begin{bmatrix} 2 \\ 1.762 \\ 3.150 \end{bmatrix}$$

$$h_3 \approx 0.245 \times 2 + 0.090 \times 0 + 0.665 \times 4 = 3.150$$

Self-Attention: The Full Computation

One query, key and value number per token is enough to see the flow.

Compare

$$S = QK^T / \sqrt{d_k}$$

$$\begin{bmatrix} 1 & 0 & 2 \\ 2 & 0 & 4 \\ 1 & 0 & 2 \end{bmatrix}$$

Compare queries and keys.

Mask + normalize

$$A = \text{softmax}(S + M)$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0.881 & 0.119 & 0 \\ 0.245 & 0.090 & 0.665 \end{bmatrix}$$

Weights over tokens, not class probabilities.

Mix

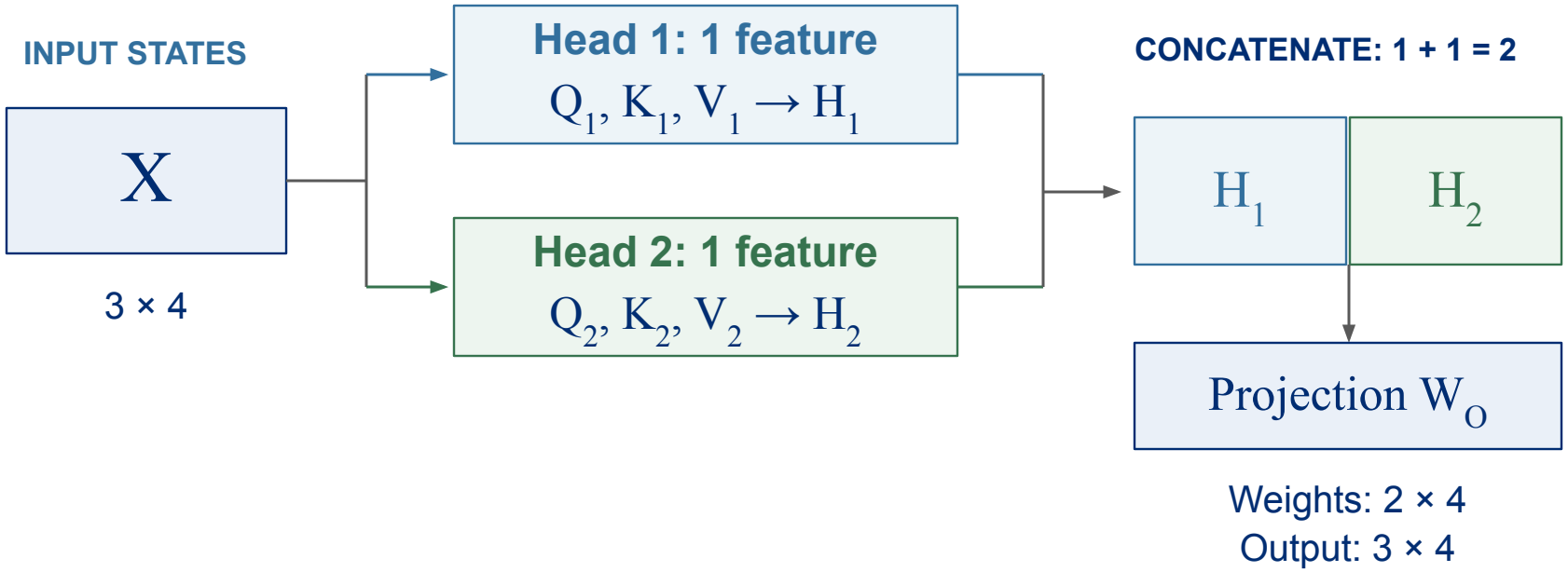
$$H = AV$$

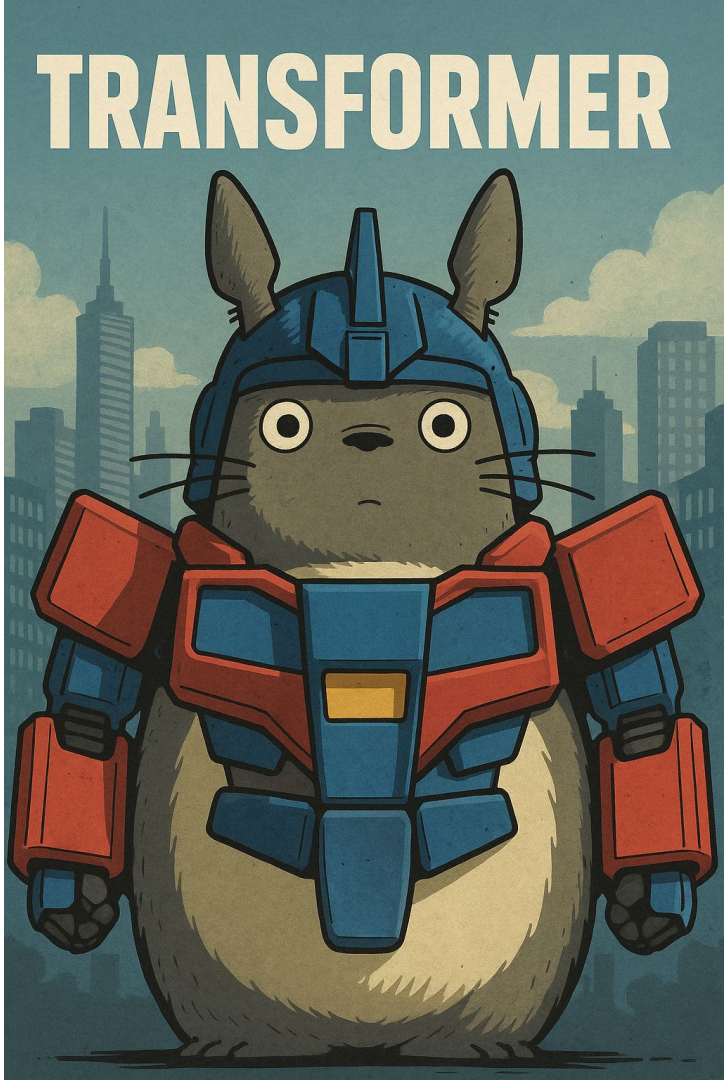
$$\begin{bmatrix} 2 \\ 1.762 \\ 3.150 \end{bmatrix}$$

3 tokens, 1 output number each.

Multi-Head Attention

A head is one attention calculation with its own learned weights.




TRANSFORMER



Residual Connections

Why Deep Models Struggle 🤯



As networks grow deeper, information and gradients can vanish or explode

→ **Early layers** might stop learning, or updates can become unstable

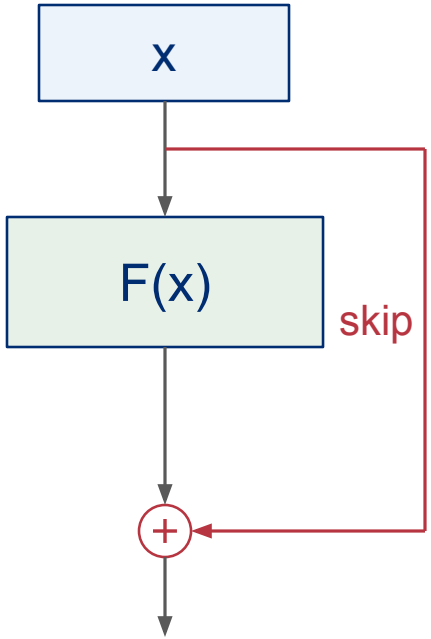
When signals vanish or explode, the model forgets what earlier layers learned, slowing convergence or breaking training

We need a way to keep useful information flowing through all layers

→ this motivates the idea of residual/skip connections

Residuals and Normalization

- Attention mixes information across positions.
- The feed-forward layer transforms each token.
- A residual path adds the input back.
- Normalization keeps layer inputs well scaled.



$$y = x + F(x)$$



Transformer Block

Putting the Pieces Together 🧠



We now have 3 key components:

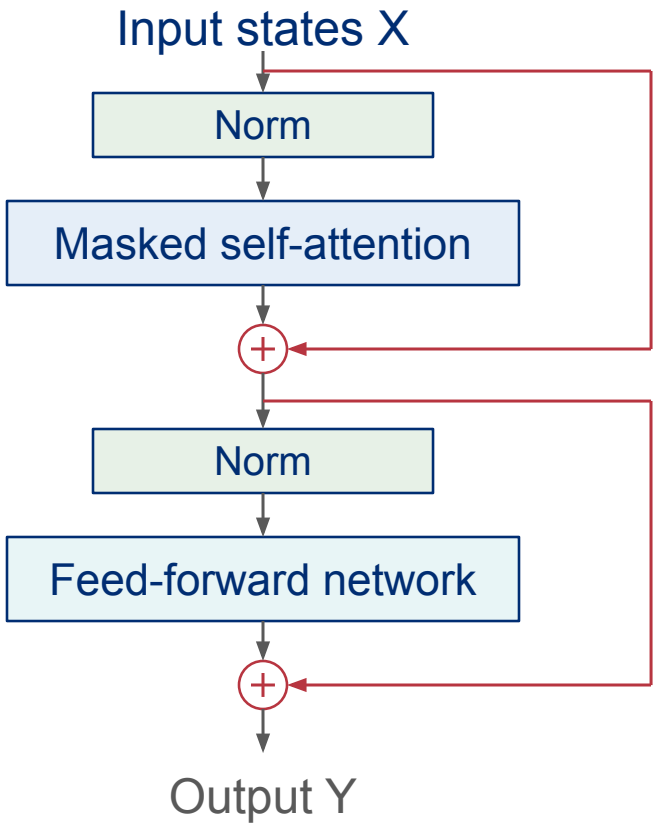
1️⃣ Self-Attention → lets tokens share information

2️⃣ Residual Connection → keeps original signal flowing

3️⃣ Feed-Forward Network → add non-linearity

Combine them → a single Transformer Block, the building unit of 🤖

Inside One Transformer Block



Residual 1

$H = X + \text{Attention}(\text{Norm}(X))$

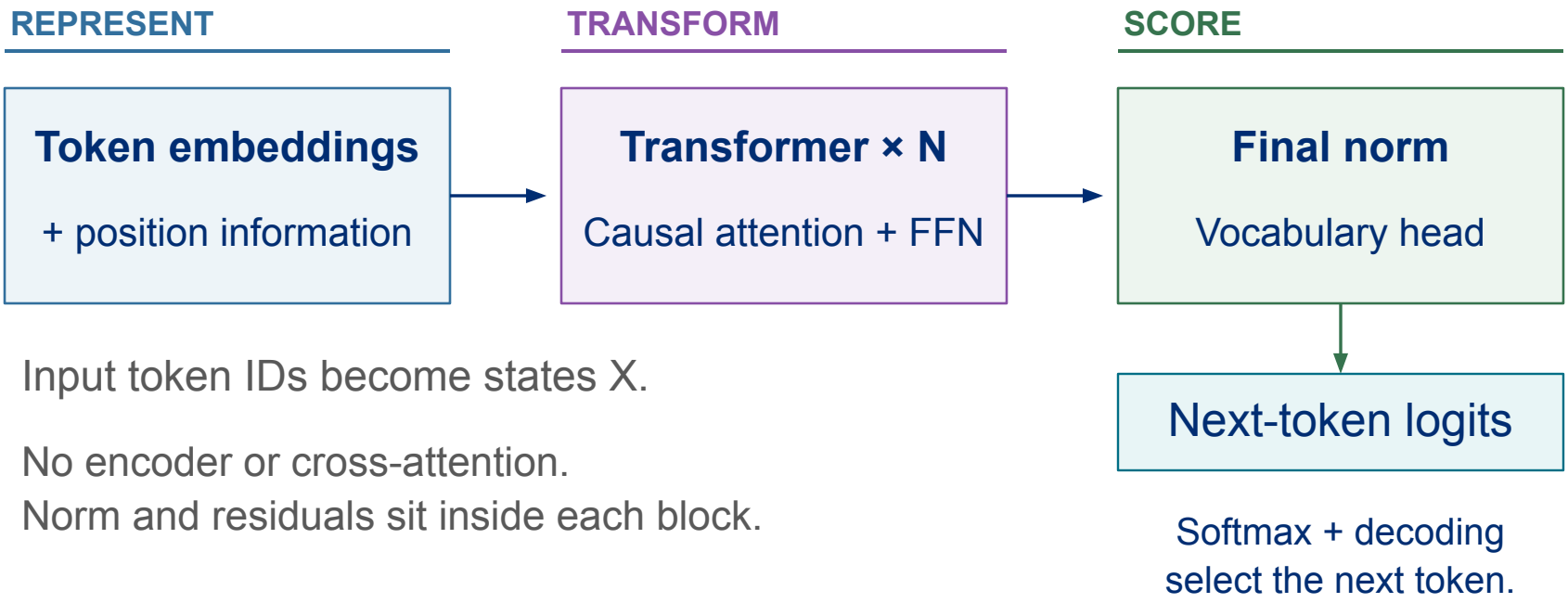
Residual 2

$Y = H + \text{FFN}(\text{Norm}(H))$

Pre-norm variant. Shape stays $T \times d_{model} = 3 \times 4$ here.

From a Block to a Decoder-Only LLM

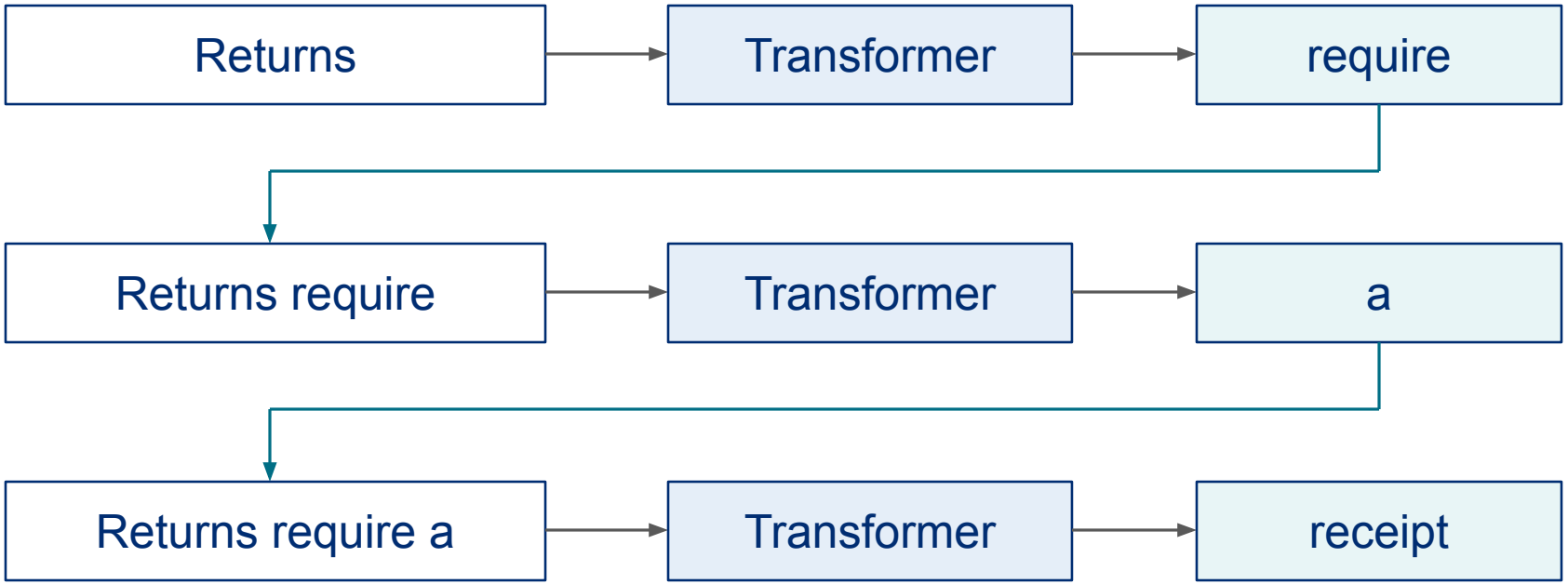
Keep the causal decoder stack; transform the available prefix.



Input token IDs become states X.

No encoder or cross-attention.
Norm and residuals sit inside each block.

Autoregressive Generation

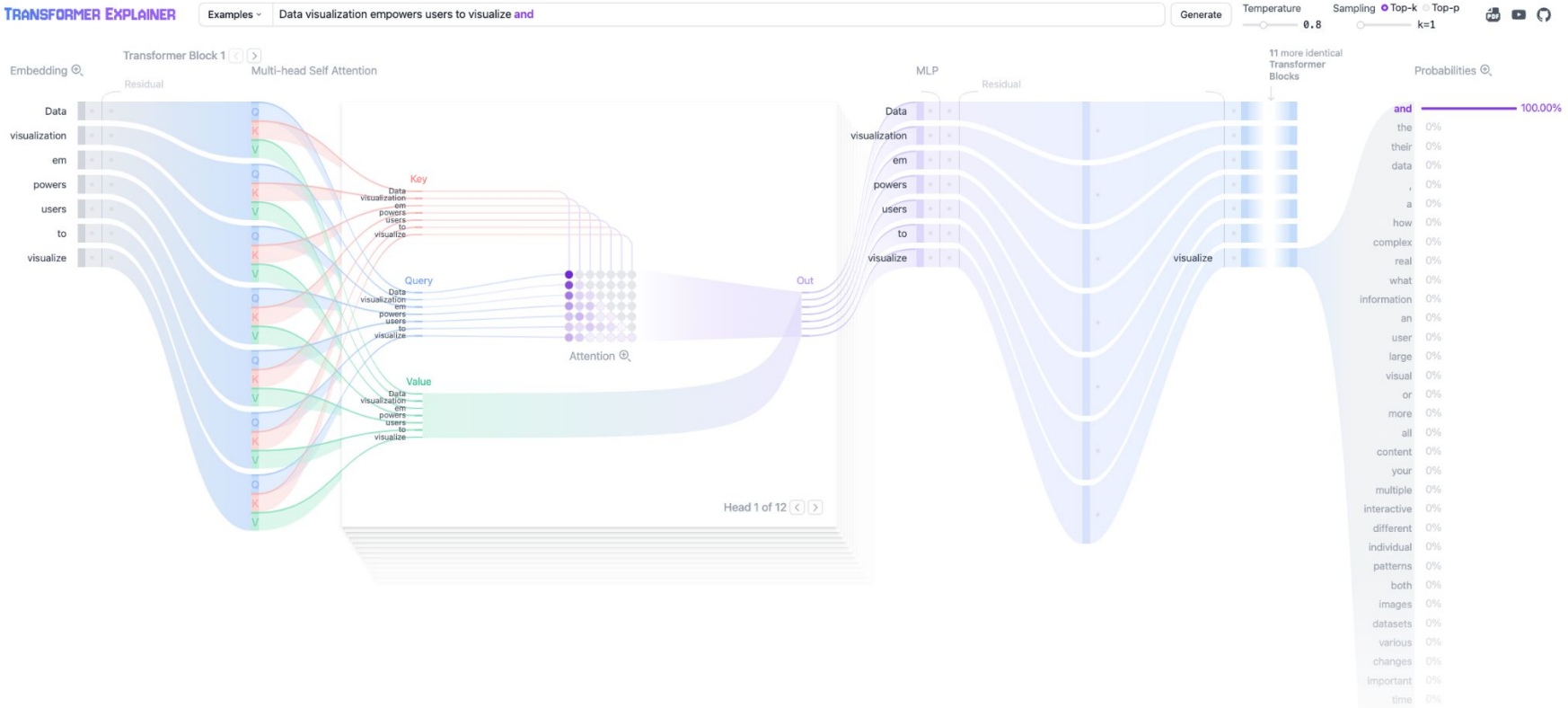


Append the selected token to the prefix, then run the next step.

Visualize Transformer 🤖



https://poloclub.github.io/transformer-explaine



Industrial Frontier



Long-Context

Long-Context Transformers 📏



Problem

Standard attention cost is $O(n^2)$ → too slow for long documents or sessions

Motivation

- Modern LLMs need to reason across 10k+ tokens: long conversations, multi-document reasoning, full-context summarization
- Long context enables more natural interactions and persistent memory

Common solutions

Sliding-Window attention, Sparse attention

Sliding Window Attention



A fixed window keeps attention cost linear in sequence length

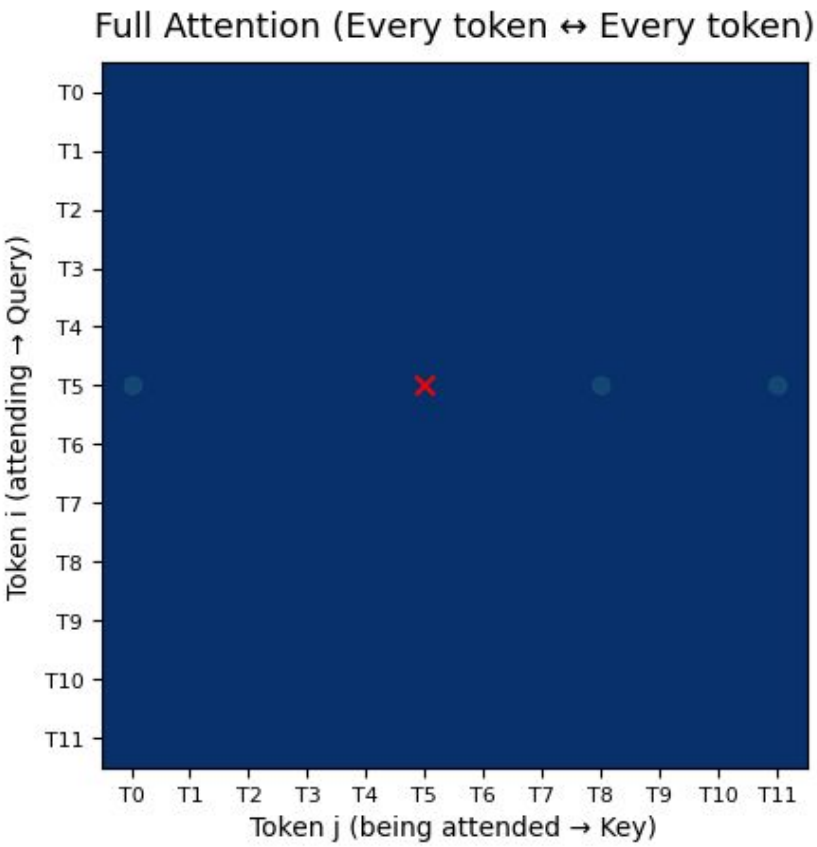
Each token attends only to nearby tokens within a fixed window size

Stacking layers lets information travel beyond one local window

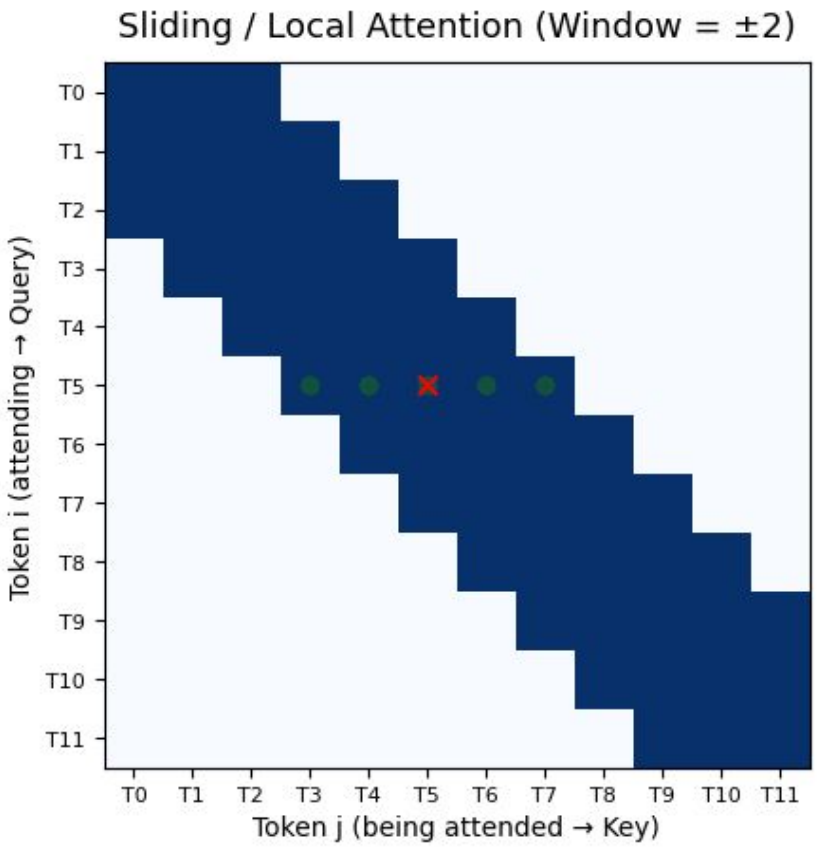
Great for long documents - balances efficiency and context

Classic examples: Longformer (2020) and Mistral 7B (2023)

Each token attends to ALL others → O(n²)



Each token attends only nearby tokens → O(n·w)



Sparse Attention ⚡



Core Idea:

- Standard attention connects every token with every other token → $O(n^2)$ cost
- Sparse attention keeps only the important links — making it much faster

How It Works:

- Local pattern: each token attends only to its neighbors
- Global pattern: a few special tokens (e.g., [CLS], separators) can see everything → summarize or broadcast information
- Random / Strided pattern: connect to a few distant tokens → let the model “jump” occasionally for long-range info

Evaluating Long Context Models 🔎



Benchmarks

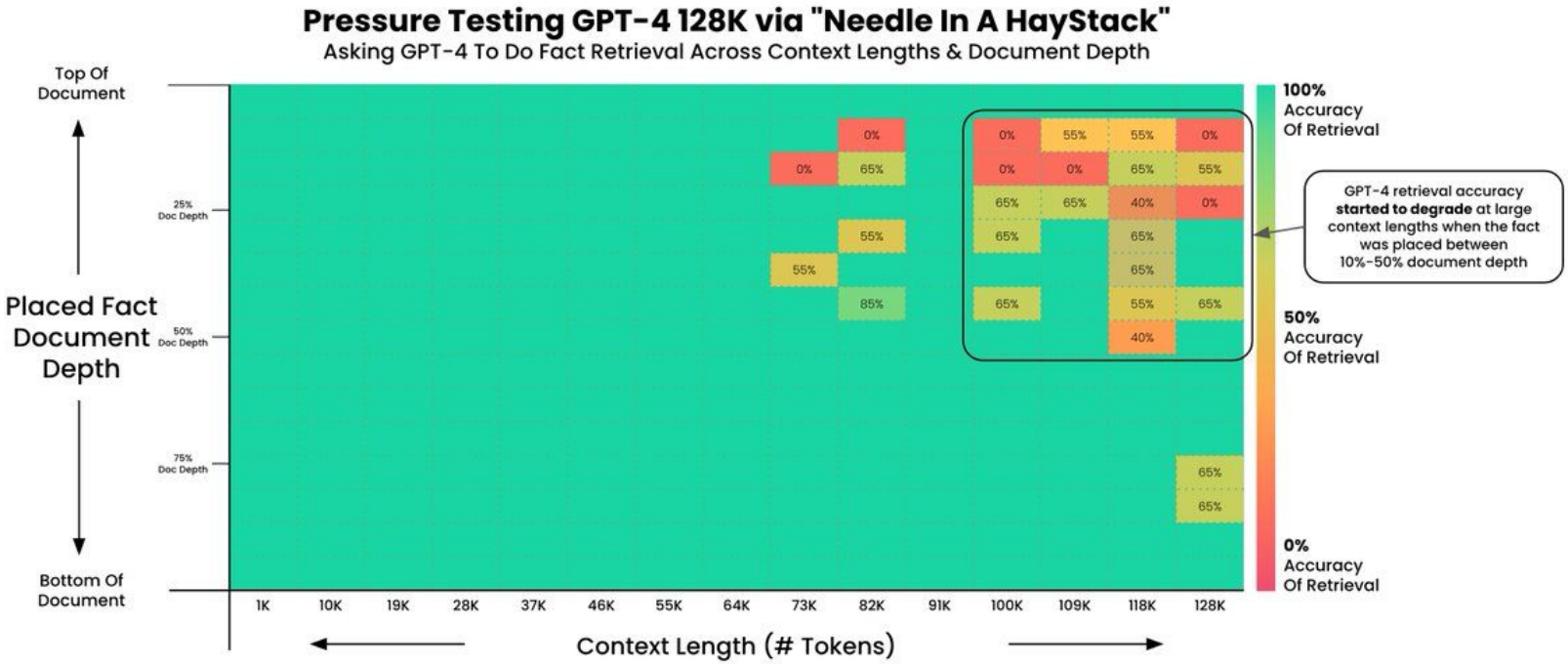
Needle-in-a-Haystack: insert a single fact in thousands of irrelevant tokens

What to Measure

Retrieval Accuracy: can the model locate the right span or entity inside 10k+ tokens?

Faithful Generation: does it use retrieved information correctly without hallucinating?

Evaluating Long Context Models 🔍



Goal: Test GPT-4 Ability To Retrieve Information From Large Context Windows

A fact was placed within a document. GPT-4 (1106-preview) was then asked to retrieve it. The output was evaluated for accuracy.
This test was run at 15 different document depths (top > bottom) and 15 different context lengths (1K >128K tokens).
2x tests were run for larger contexts for a larger sample size.



Mixture-of-Experts (MoE)

Mixture-of-Experts (MoE) 🧠



Each token activates only a small subset (top-k) of experts

More total parameters at similar active compute; routing still costs

Router network decides which experts process each token

Experts are trained jointly and specialize on different patterns

Examples: Switch Transformer, Mixtral, and DeepSeek MoE models

Routing and Balancing ⚖️

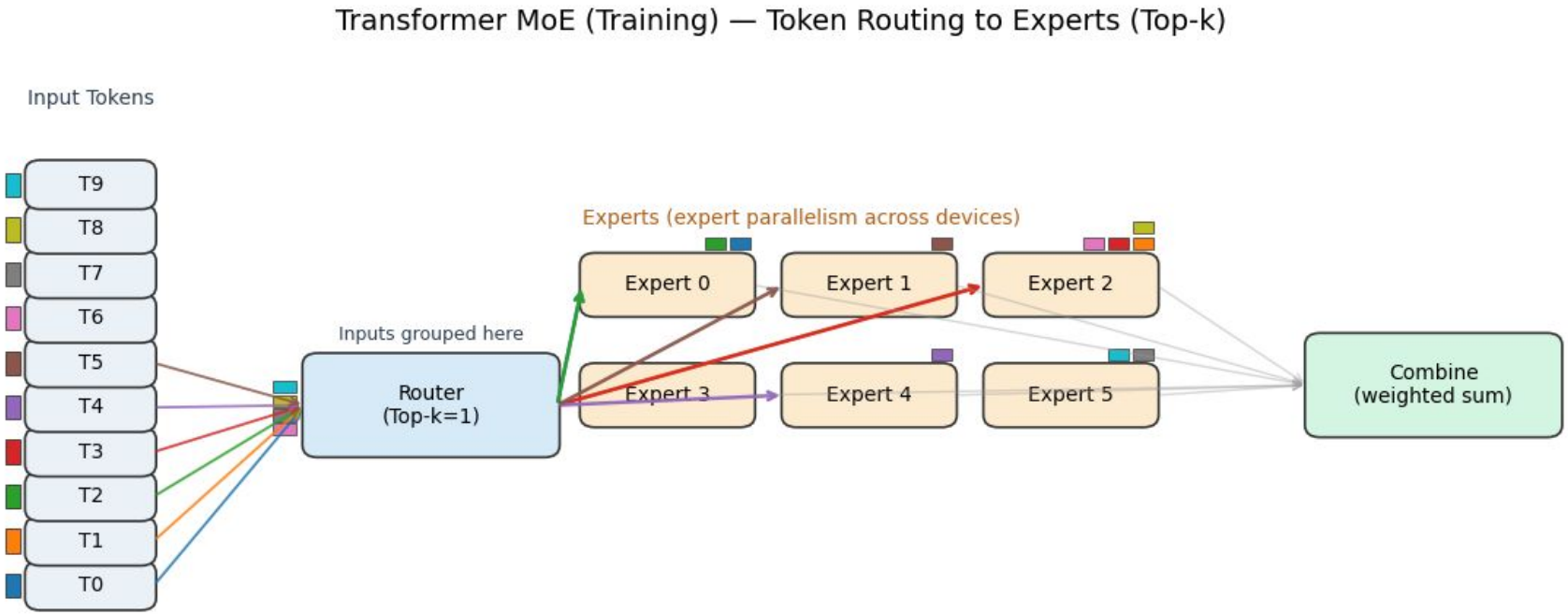


Router predicts which experts to use for each token

- Predicts which experts each token should use (usually top-1 or top-2)
- Implemented as a lightweight gating network - outputs routing probabilities

Load balancing loss prevents all tokens from choosing same experts

- Ensures tokens are distributed evenly across experts
- Prevents “expert collapse,” where all tokens pick the same expert


Transformer MoE (Training) — Token Routing to Experts (Top-k)
Input Tokens
T9
T8
T7
T6
T5
T4
T3
T2
T1
T0
Inputs grouped here
Router
(Top-k=1)
Experts (expert parallelism across devices)
Expert 0
Expert 1
Expert 2
Expert 3
Expert 4
Expert 5
Combine
(weighted sum)

AI of the Week: Efficient Long Context

DeepSeek V4.1 Flash (September 2026)

- Combines sparse attention and MoE with a 1M-token context window.
- Reuses attention state across layers to reduce KV-cache memory.

Connect it to today's lecture

- Sparse attention limits connections; MoE limits active experts.
- A large context window still needs retrieval-quality evaluation.

Official model card; API update Oct 17. Checked Oct 23, 2026.

Summary



Next-Token Prediction: Predicts the next token from previous ones

Self-Attention: Shares context among tokens

Residual Connection: Keeps information flowing, stabilizes training

Feed-Forward Network: Enhances non-linearity to each token

Transformer Block: Combines all 3 - The core unit of modern LLMs

Industrial Frontier: Long context + Sparse Transformer